

Lembar Kegiatan Perkuliahan PBO

Identitas Kegiatan

- **Mata Kuliah:** Pemrograman Berorientasi Objek
- **Topik:** Perancangan Class dan Relasi Antar Class (Association)
- **Alokasi waktu:** 3×50 menit
- **Model Pembelajaran:** Scientific Computational Thinking (SCT)
- **Bentuk Kegiatan:** Kelompok (5–6 mahasiswa)

Kelompok Kerja

- Irsyad Maulana (K3525008)
- Haikal Wilhan
- Muh Rizky Daniswara (K3525033)
- Niceforus Helas (K3525037)
- Fathaya Khairan

Capaian Pembelajaran

Mahasiswa mampu merancang solusi permasalahan dengan menerapkan pendekatan PBO sederhana (membuat struktur class yang terdiri dari atribut dan method), serta relasi antar class (tanpa inheritance), dan mengimplementasikan program berdasarkan rancangan solusi yang dibuat.

Tujuan Kegiatan

Setelah mengikuti kegiatan ini, mahasiswa mampu:

1. Mengidentifikasi masalah dan komponen objek dalam suatu kasus.
2. Menganalisis kebutuhan sistem dan hubungan antar objek.
3. Merancang class beserta atribut dan method secara tepat.
4. Menentukan relasi antar class (association).
5. Mengimplementasikan solusi dalam bentuk program Python.
6. Mengevaluasi dan mengkomunikasikan solusi yang dibuat.

Petunjuk Kegiatan

1. Kerjakan secara berkelompok.
2. Ikuti tahapan kegiatan sesuai model SCT.
3. Diskusikan setiap jawaban sebelum dituliskan.
4. Implementasikan solusi setelah rancangan disepakati.
5. Presentasikan hasil kerja kelompok.

Studi Kasus

Sebuah klinik hewan ingin mengembangkan sistem sederhana untuk membantu pengelolaan data hewan peliharaan serta layanan dokter. Dalam sistem ini, setiap pemilik dapat memiliki satu atau lebih hewan peliharaan yang terdaftar di klinik. Setiap hewan memiliki informasi dasar seperti nama, jenis, umur, dan kondisi kesehatan singkat. Selain itu, setiap hewan dapat ditangani oleh seorang dokter hewan yang memiliki data identitas serta bidang spesialisasi tertentu. Untuk mendukung proses perawatan, klinik juga menyediakan ruang perawatan yang masing-masing memiliki kode ruang, nama ruang, dan kapasitas tertentu, di mana setiap hewan dapat ditempatkan pada salah satu ruang tersebut. Sistem yang dikembangkan diharapkan mampu menampilkan informasi secara lengkap mengenai pemilik, hewan peliharaan, dokter yang menangani, serta ruang perawatan yang digunakan.

Berikut ini adalah daftar tarif periksa hewan:

- Tanpa rawat inap: sesuai tarif periksa masing-masing dokter
- Dengan rawat inap: tarif periksa masing-masing dokter + rawat inap Rp 200.000/hari

Sistem diharapkan juga dapat digunakan untuk menghitung total tarif periksa hewan.

A. Scoping

Tugas

Identifikasi komponen penting dari permasalahan.

Aspek	Jawaban
Masalah utama	Tidak adanya sistem digital untuk manajemen klinik hewan.
Entitas apa yang terlibat	Pemilik, Dokter, Hewan, dan Ruang.
Data yang perlu disimpan	Pemilik: ID Pemilik, Nama Pemilik, Narahubung Pemilik Hewan: ID Hewan, Nama Hewan, Jenis, Umur, Kondisi Kesehatan, Status Rawat Inap Dokter: ID Dokter, Nama Dokter, Spesialisasi Dokter, Narahubung Dokter, Tarif dokter Ruang: ID Ruang, Nama Ruang, Kapasitas
Proses yang harus dilakukan	1. Input informasi (informasi untuk pemilik, hewan, dokter, dan ruang) 2. Simpan informasi di setiap objek 3. Kalkulasi tarif berdasarkan status rawat inap dan tarif dokter 4. Output kalkulasi berupa <i>invoice</i>
Output yang diharapkan	Dapat menampilkan: 1. Data yang disimpan 2. Tarif hewan

B. Questioning

Tulislah pertanyaan-pertanyaan yang akan membantu merancang solusi.

1. Class apa saja yang perlu dibuat?
2. Atribut apa yang dimiliki setiap class?
3. Method apa yang dibutuhkan?
4. Apa relasi antar Class?
5. Bagaimana data disimpan?
6. Bagaimana proses *indexing*?
7. Bagaimana *output* ditampilkan?

C. Design Thinking

Rancangan Class

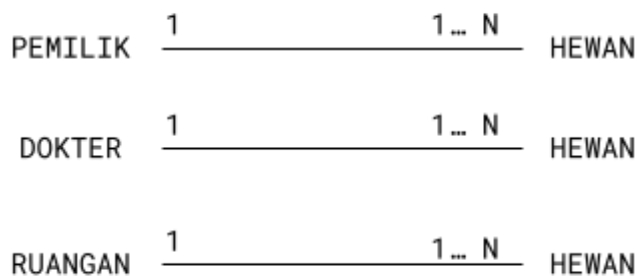
Berdasarkan entitas yang terlibat dalam sistem, tulislah class apa saja yang berpotensi untuk dibuat

Class	Atribut	Method	Deskripsi
Pemilik	id_pemilik, nama_pemilik, narahubung_pemilik, daftar_hewan	<code>__init__</code> , <code>getData()</code>	- <code>__init__(self, **kwargs)</code>: digunakan untuk menerima input dan menyimpan informasi tentang pemilik - <code>getData(self, query=None)</code>: digunakan untuk <i>indexing</i> pemilik
Hewan	id_hewan, nama_hewan, jenis_hewan, kondisi_hewan, status_rawat_inap, dokter_hewan, ruangan	<code>__init__</code> , <code>getData()</code> , <code>saveData()</code>	- <code>__init__(self, **kwargs)</code>: digunakan untuk menerima input dan menyimpan informasi tentang hewan - <code>getData(self, query=None)</code>: digunakan untuk <i>indexing</i> hewan - <code>saveData(self, query=None, input)</code>: digunakan untuk mengubah/menyimpan informasi hewan
Dokter	id_dokter, nama_dokter, spesialis_dokter, narahubung_dokter, tarif_dokter	<code>__init__</code> , <code>getData()</code> , <code>assignPet()</code>	- <code>__init__(self, **kwargs)</code>: digunakan untuk menerima input dan menyimpan informasi tentang dokter - <code>getData(self, query=None)</code>: digunakan untuk <i>indexing</i> informasi dokter - <code>assignPet(self, query=None, input)</code>: digunakan untuk menambah pasien

Class	Atribut	Method	Deskripsi
Ruang	id_ruang, nama_ruang, kapasitas_ruang	<u>init</u> , getData()	- <code>__init__(self, **kwargs)</code>: digunakan untuk menerima input dan menyimpan informasi tentang ruangan - <code>getData(self, query=None)</code>: digunakan untuk menerima input dan menyimpan informasi tentang hewan.

Relasi Antar Class

Berdasarkan rancangan class dari entitas sebelumnya, tentukan class-class yang saling berelasi.



Justifikasi Solusi

Berdasarkan rancangan class dari setiap anggota kelompok, selanjutnya diskusikan dalam kelompok untuk menentukan rancangan mana yang terbaik dari anggota kelompok.

Rancangan Class Terbaik

```

class Ruang:
    _data = {}

    def __init__(self, **kwargs):
        self.id_ruang = kwargs.get("id_ruang")
        self.nama_ruang = kwargs.get("nama_ruang")
        self.kapasitas_ruang = kwargs.get("kapasitas_ruang")
        Ruang._data[self.id_ruang] = self

    def getData(self, query=None):
        if query is None:
            return Ruang._data
        return Ruang._data.get(query)

    def __str__(self):
        return (f"Ruang : {self.nama_ruang} "
                f"(ID: {self.id_ruang}, Kapasitas: {self.kapasitas_ruang})")
  
```

```

class Dokter:
    _data = {}

    def __init__(self, **kwargs):
        self.id_dokter = kwargs.get("id_dokter")
        self.nama_dokter = kwargs.get("nama_dokter")
        self.spesialis_dokter = kwargs.get("spesialis_dokter")
        self.narahubung_dokter = kwargs.get("narahubung_dokter")
        self.tarif_dokter = kwargs.get("tarif_dokter", 0)
        self.daftar_pasien = []
        Dokter._data[self.id_dokter] = self

    def getData(self, query=None):
        if query is None:
            return Dokter._data
        return Dokter._data.get(query)

    def assignPet(self, query=None, input=None):
        target_dokter = Dokter._data.get(query) if query else self
        if target_dokter and input:
            target_dokter.daftar_pasien.append(input)
            print(f"[assignPet] Hewan '{input}' ditambahkan ke "
                  f"dr. {target_dokter.nama_dokter}.")

    def __str__(self):
        return (f"Dokter      : {self.nama_dokter} "
                f"(Spesialis: {self.spesialis_dokter}, "
                f"Tarif: Rp {self.tarif_dokter:,})")

class Hewan:
    _data = {}

    def __init__(self, **kwargs):
        self.id_hewan = kwargs.get("id_hewan")
        self.nama_hewan = kwargs.get("nama_hewan")
        self.jenis_hewan = kwargs.get("jenis_hewan")
        self.kondisi_hewan = kwargs.get("kondisi_hewan")
        self.status_rawat_inap = kwargs.get("status_rawat_inap", False)
        self.lama_rawat_inap = kwargs.get("lama_rawat_inap", 0)
        self.dokter_hewan = kwargs.get("dokter_hewan")
        self.ruangan = kwargs.get("ruangan")
        Hewan._data[self.id_hewan] = self

    def getData(self, query=None):
        """Indexing hewan."""
        if query is None:

```

```

        return Hewan._data
    return Hewan._data.get(query)

def saveData(self, query=None, input=None):
    target = Hewan._data.get(query) if query else self
    if target and isinstance(input, dict):
        for key, value in input.items():
            setattr(target, key, value)
        print(f"[saveData] Data hewan '{target.id_hewan}' diperbarui.")

def hitung_tarif(self):
    TARIF_INAP_PER_HARI = 200_000
    dokter_obj = Dokter._data.get(self.dokter_hewan)
    tarif_dokter = dokter_obj.tarif_dokter if dokter_obj else 0

    if self.status_rawat_inap:
        total = tarif_dokter + (TARIF_INAP_PER_HARI * self.lama_rawat_inap)
    else:
        total = tarif_dokter
    return total

def __str__(self):
    return (f"Hewan          : {self.nama_hewan} "
            f"(ID: {self.id_hewan}, Jenis: {self.jenis_hewan}, "
            f"Kondisi: {self.kondisi_hewan})")

class Pemilik:
    _data = {}

    def __init__(self, **kwargs):
        self.id_pemilik      = kwargs.get("id_pemilik")
        self.nama_pemilik    = kwargs.get("nama_pemilik")
        self.narahubung_pemilik = kwargs.get("narahubung_pemilik")
        self.daftar_hewan    = kwargs.get("daftar_hewan", [])
        Pemilik._data[self.id_pemilik] = self

    def getData(self, query=None):
        """Indexing pemilik."""
        if query is None:
            return Pemilik._data
        return Pemilik._data.get(query)

    def tampilkan_info_lengkap(self):
        TARIF_INAP_PER_HARI = 200_000
        print("=" * 60)
        print(f"PEMILIK      : {self.nama_pemilik}")
        print(f"ID Pemilik   : {self.id_pemilik}")

```

```

print(f"Narahubung : {self.narahubung_pemilik}")
print("-" * 60)

for id_hw in self.daftar_hewan:
    hewan = Hewan._data.get(id_hw)
    if not hewan:
        continue

    print(str(hewan))
    print(f"    Kondisi      : {hewan.kondisi_hewan}")

    dokter = Dokter._data.get(hewan.dokter_hewan)
    if dokter:
        print(f"        {dokter}")
        print(f"    Narahubung Dokter : {dokter.narahubung_dokter}")
    else:
        print("    Dokter          : (tidak ada)")

    ruang = Ruang._data.get(hewan.ruangan)
    if ruang:
        print(f"        {ruang}")
    else:
        print("    Ruang            : (tidak ada / tidak rawat inap)")

    status_str = (
        f"Rawat Inap ({hewan.lama_rawat_inap} hari)"
        if hewan.status_rawat_inap else "Tanpa Rawat Inap"
    )
    total = hewan.hitung_tarif()
    tarif_dokter = dokter.tarif_dokter if dokter else 0

    print(f"    Status          : {status_str}")
    print(f"    Rincian Tarif:")
    print(f"        - Tarif Dokter      : Rp {tarif_dokter:>10,}")
    if hewan.status_rawat_inap:
        biaya_inap = TARIF_INAP_PER_HARI * hewan.lama_rawat_inap
        print(f"        - Biaya Rawat Inap  : Rp {biaya_inap:>10,} "
              f"(Rp 200.000 × {hewan.lama_rawat_inap} hari)")
    print(f"        _____")
    print(f"    TOTAL              : Rp {total:>10,}")
    print("-" * 60)

print("=" * 60)

def __str__(self):
    return (f"Pemilik: {self.nama_pemilik} "
            f"(ID: {self.id_pemilik}, "
            f"HP: {self.narahubung_pemilik})")

```

Alasannya

Pendekatan ini lebih efisien dan efektif karena setiap kelas bersifat independen, dengan penyimpanan data dan behaviour (method) masing-masing. Hal ini memungkinkan relasi yang lebih efektif karena setiap instansi memiliki ID yang menjadi data pengenal. Secara keterbacaan, kode ini lebih rapi dan bersih (*clean*), sehingga memiliki skalabilitas tinggi dan mudah dibaca/dipahami. Setiap inisialisasi menggunakan tipe data *dictionary* yang memudahkan proses *indexing* ketika mencari dan memvalidasi data.